

手摸手实现公共组件库

组件地址

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-common-spring-boot-starter</artifactId>
  <version>${project.version}</version>
</dependency>
```

组件概述

1. 抽象常用枚举码值，比如：删除标记枚举、标识枚举、操作类型以及状态枚举等。
2. 封装线程池相关功能，比如：线程池构造器、快速执行线程池、动态代理拒绝策略等。
3. 常用工具类，比如：断言类、对象复制工具类、环境类以及线程工具类等。

组件功能

抽象常用枚举

枚举在日常开发项目中占着比较重要的位置，有一些每个项目都要用到的枚举可以尝试抽象出来，避免重复创建。比如说，逻辑删除枚举等。

```
package org.opengoofy.index12306.framework.starter.common.enums;

/**
 * 删除标记枚举
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public enum DelEnum {

    /**
     * 正常状态
```

```

        */
        NORMAL(0),

        /**
         * 删除状态
         */
        DELETE(1);

        private final Integer statusCode;

        DelEnum(Integer statusCode) {
            this.statusCode = statusCode;
        }

        public Integer code() {
            return this.statusCode;
        }

        public String strCode() {
            return String.valueOf(this.statusCode);
        }

        @Override
        public String toString() {
            return strCode();
        }
    }
}

```

封装线程池

线程池封装这里，有两个比较有意思的功能。一个是封装快速消费线程池，另一个就是通过动态代理实现拒绝策略的扩展。

1) 快速消费线程池。

首先定义快速消费线程池的阻塞队列，不能使用原有的常用队列。

```

package org.opengoofy.index12306.framework.starter.common.threadpool.support.eager;

import lombok.Setter;

```

```
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.TimeUnit;

/**
 * 快速消费任务队列
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public class TaskQueue<R extends Runnable> extends LinkedBlockingQueue<Runnable> {

    @Setter
    private EagerThreadPoolExecutor executor;

    public TaskQueue(int capacity) {
        super(capacity);
    }

    @Override
    public boolean offer(Runnable runnable) {
        int currentPoolThreadSize = executor.getPoolSize();
        // 如果有核心线程正在空闲，将任务加入阻塞队列，由核心线程进行处理任务
        if (executor.getSubmittedTaskCount() < currentPoolThreadSize) {
            return super.offer(runnable);
        }
        // 当前线程池线程数量小于最大线程数，返回 False，根据线程池源码，会创建非核心线程
        if (currentPoolThreadSize < executor.getMaximumPoolSize()) {
            return false;
        }
        // 如果当前线程池数量大于最大线程数，任务加入阻塞队列
        return super.offer(runnable);
    }

    public boolean retryOffer(Runnable o, long timeout, TimeUnit unit) throws
    InterruptedException {
        if (executor.isShutdown()) {
            throw new RejectedExecutionException("Executor is shutdown!");
        }
        return super.offer(o, timeout, unit);
    }
}
```

```
}  
}
```

定义快速消费线程池，这里参考了 Dubbo 的线程池模型之一。另外，Tomcat 也是这种消费模型。

```
package org.opengoofy.index12306.framework.starter.common.threadpool.support.eager;  
  
import java.util.concurrent.RejectedExecutionException;  
import java.util.concurrent.RejectedExecutionHandler;  
import java.util.concurrent.ThreadFactory;  
import java.util.concurrent.ThreadPoolExecutor;  
import java.util.concurrent.TimeUnit;  
import java.util.concurrent.atomic.AtomicInteger;  
  
/**  
 * 快速消费线程池  
 *  
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料  
 */  
public class EagerThreadPoolExecutor extends ThreadPoolExecutor {  
  
    public EagerThreadPoolExecutor(int corePoolSize,  
                                   int maximumPoolSize,  
                                   long keepAliveTime,  
                                   TimeUnit unit,  
                                   TaskQueue<Runnable> workQueue,  
                                   ThreadFactory threadFactory,  
                                   RejectedExecutionHandler handler) {  
        super(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue,  
threadFactory, handler);  
    }  
  
    private final AtomicInteger submittedTaskCount = new AtomicInteger(0);  
  
    public int getSubmittedTaskCount() {  
        return submittedTaskCount.get();  
    }  
  
    @Override
```

```

protected void afterExecute(Runnable r, Throwable t) {
    submittedTaskCount.decrementAndGet();
}

@Override
public void execute(Runnable command) {
    submittedTaskCount.incrementAndGet();
    try {
        super.execute(command);
    } catch (RejectedExecutionException ex) {
        TaskQueue taskQueue = (TaskQueue) super.getQueue();
        try {
            if (!taskQueue.retryOffer(command, 0, TimeUnit.MILLISECONDS)) {
                submittedTaskCount.decrementAndGet();
                throw new RejectedExecutionException("Queue capacity is full.", ex);
            }
        } catch (InterruptedException iex) {
            submittedTaskCount.decrementAndGet();
            throw new RejectedExecutionException(iex);
        }
    } catch (Exception ex) {
        submittedTaskCount.decrementAndGet();
        throw ex;
    }
}
}

```

如果不容易理解，可以查看下述文档掌握快速线程池的设计要点。

[参考Dubbo线程池模型实现快速消费线程池](#)

2) 动态代理扩展线程池拒绝策略。

定义动态代理类。

```

package org.opengoofy.index12306.framework.starter.common.threadpool.proxy;

import lombok.AllArgsConstructor;
import lombok.extern.slf4j.Slf4j;

import java.lang.reflect.InvocationHandler;

```

```

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.concurrent.atomic.AtomicLong;

/**
 * 线程池拒绝策略代理执行器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j
@AllArgsConstructor
public class RejectedProxyInvocationHandler implements InvocationHandler {

    /**
     * Target object
     */
    private final Object target;

    /**
     * Reject count
     */
    private final AtomicLong rejectCount;

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        rejectCount.incrementAndGet();
        try {
            log.error("线程池执行拒绝策略，此处模拟报警...");
            return method.invoke(target, args);
        } catch (InvocationTargetException ex) {
            throw ex.getCause();
        }
    }
}

```

每次去创建动态代理类的方法也比较麻烦，我们提供一个动态代理类创建的工具类。

并附带单元测试。

```

package org.opengoofy.index12306.framework.starter.common.threadpool.proxy;

```

```

import lombok.AccessLevel;
import lombok.NoArgsConstructor;
import org.opengoofy.index12306.framework.starter.common.toolkit.ThreadUtil;

import java.lang.reflect.Proxy;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.RejectedExecutionHandler;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

/**
 * 拒绝策略代理工具类
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@NoArgsConstructor(access = AccessLevel.PRIVATE)
public final class RejectedProxyUtil {

    /**
     * 创建拒绝策略代理类
     *
     * @param rejectedExecutionHandler 真正的线程池拒绝策略执行器
     * @param rejectedNum 拒绝策略执行统计器
     * @return 代理拒绝策略
     */
    public static RejectedExecutionHandler createProxy(RejectedExecutionHandler
rejectedExecutionHandler, AtomicLong rejectedNum) {
        // 动态代理模式：增强线程池拒绝策略，比如：拒绝任务报警或加入延迟队列重复放入等逻辑
        return (RejectedExecutionHandler) Proxy
            .newProxyInstance(
                rejectedExecutionHandler.getClass().getClassLoader(),
                new Class[]{RejectedExecutionHandler.class},
                new RejectedProxyInvocationHandler(rejectedExecutionHandler,
rejectedNum));
    }

    /**
     * 测试线程池拒绝策略动态代理程序
     */
    public static void main(String[] args) {

```

```

        ThreadPoolExecutor threadPoolExecutor =
            new ThreadPoolExecutor(1, 3, 1024, TimeUnit.SECONDS, new
LinkedBlockingQueue<>(1));
        ThreadPoolExecutor.AbortPolicy abortPolicy = new
ThreadPoolExecutor.AbortPolicy();
        AtomicLong rejectedNum = new AtomicLong();
        RejectedExecutionHandler proxyRejectedExecutionHandler =
RejectedProxyUtil.createProxy(abortPolicy, rejectedNum);
        threadPoolExecutor.setRejectedExecutionHandler(proxyRejectedExecutionHandler);
        for (int i = 0; i < 5; i++) {
            try {
                threadPoolExecutor.execute(() -> ThreadUtil.sleep(100000L));
            } catch (Exception ignored) {
                ignored.printStackTrace();
            }
        }
        System.out.println("===== 线程池拒绝策略执行次数: " +
rejectedNum.get());
    }
}

```

如果大家看代码不容易理解，这里提供了详细的文章解释动态代理，以及 MyBatis 中如何用的动态代理。

[学习Mybatis动态代理扩展拒绝策略](#)

常用工具类

都是一些工具类，这里就不再赘述。

```

package org.opengoofy.index12306.framework.starter.common.toolkit;

import com.github.dozermapper.core.DozerBeanMapperBuilder;
import com.github.dozermapper.core.Mapper;
import com.github.dozermapper.core.loader.api.BeanMappingBuilder;
import lombok.NoArgsConstructor;

import java.lang.reflect.Array;
import java.util.ArrayList;
import java.util.HashSet;

```



```
import java.util.List;
import java.util.Optional;
import java.util.Set;

import static com.github.dozermapper.core.loader.api.TypeMappingOptions.mapEmptyString;
import static com.github.dozermapper.core.loader.api.TypeMappingOptions.mapNull;

/**
 * 对象属性复制工具类
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@NoArgsConstructor(access = lombok.AccessLevel.PRIVATE)
public class BeanUtil {

    protected static Mapper BEAN_MAPPER_BUILDER;

    static {
        BEAN_MAPPER_BUILDER = DozerBeanMapperBuilder.buildDefault();
    }

    /**
     * 属性复制
     *
     * @param source 数据对象
     * @param target 目标对象
     * @param <T>
     * @param <S>
     * @return 转换后对象
     */
    public static <T, S> T convert(S source, T target) {
        Optional.ofNullable(source)
            .ifPresent(each -> BEAN_MAPPER_BUILDER.map(each, target));
        return target;
    }

    /**
     * 复制单个对象
     *
     * @param source 数据对象
     * @param clazz 复制目标类型
     */
}
```

```

    * @param <T>
    * @param <S>
    * @return 转换后对象
    */
    public static <T, S> T convert(S source, Class<T> clazz) {
        return Optional.ofNullable(source)
            .map(each -> BEAN_MAPPER_BUILDER.map(each, clazz))
            .orElse(null);
    }

    /**
     * 复制多个对象
     *
     * @param sources 数据对象
     * @param clazz 复制目标类型
     * @param <T>
     * @param <S>
     * @return 转换后对象集合
     */
    public static <T, S> List<T> convert(List<S> sources, Class<T> clazz) {
        return Optional.ofNullable(sources)
            .map(each -> {
                List<T> targetList = new ArrayList<T>(each.size());
                each.stream()
                    .forEach(item -> targetList.add(BEAN_MAPPER_BUILDER.map(item,
clazz)));
                return targetList;
            })
            .orElse(null);
    }

    /**
     * 复制多个对象
     *
     * @param sources 数据对象
     * @param clazz 复制目标类型
     * @param <T>
     * @param <S>
     * @return 转换后对象集合
     */
    public static <T, S> Set<T> convert(Set<S> sources, Class<T> clazz) {

```

```

        return Optional.ofNullable(sources)
            .map(each -> {
                Set<T> targetSize = new HashSet<T>(each.size());
                each.stream()
                    .forEach(item -> targetSize.add(BEAN_MAPPER_BUILDER.map(item,
clazz)));
                return targetSize;
            })
            .orElse(null);
    }

    /**
     * 复制多个对象
     *
     * @param sources 数据对象
     * @param clazz 复制目标类型
     * @param <T>
     * @param <S>
     * @return 转换后对象集合
     */
    public static <T, S> T[] convert(S[] sources, Class<T> clazz) {
        return Optional.ofNullable(sources)
            .map(each -> {
                @SuppressWarnings("unchecked")
                T[] targetArray = (T[]) Array.newInstance(clazz, sources.length);
                for (int i = 0; i < targetArray.length; i++) {
                    targetArray[i] = BEAN_MAPPER_BUILDER.map(sources[i], clazz);
                }
                return targetArray;
            })
            .orElse(null);
    }

    /**
     * 拷贝非空且非空串属性
     *
     * @param source 数据源
     * @param target 指向源
     */
    public static void convertIgnoreNullAndBlank(Object source, Object target) {
        DozerBeanMapperBuilder dozerBeanMapperBuilder = DozerBeanMapperBuilder.create();
    }

```

```

        Mapper mapper = dozerBeanMapperBuilder.withMappingBuilders(new
BeanMappingBuilder() {

            @Override
            protected void configure() {
                mapping(source.getClass(), target.getClass(), mapNull(false),
mapEmptyString(false));
            }
        }).build();
        mapper.map(source, target);
    }

    /**
     * 拷贝非空属性
     *
     * @param source 数据源
     * @param target 指向源
     */
    public static void convertIgnoreNull(Object source, Object target) {
        DozerBeanMapperBuilder dozerBeanMapperBuilder = DozerBeanMapperBuilder.create();
        Mapper mapper = dozerBeanMapperBuilder.withMappingBuilders(new
BeanMappingBuilder() {

            @Override
            protected void configure() {
                mapping(source.getClass(), target.getClass(), mapNull(false));
            }
        }).build();
        mapper.map(source, target);
    }
}

```

更新: 2023-07-17 15:58:25

原文: <<https://www.yuque.com/magestack/12306/fh5oa773pd8mtduw>>